

SIMATS ENGINEERING
CO3 - ASSESSMENT TOOL - 2: Case Study

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Name	Buvanesh Y
Register Number	192372367

COURSE OUTCOME COVERED

CO3: *Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications. (BL3)*

Dave's Taxonomy: Precision (Level 3)
Question Count: 3

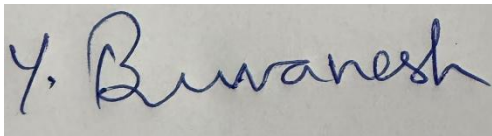
Total Marks: 30

Code of Conduct

I **Buvanesh Y (192372367)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Assessment Tasks - Case Study

1. Case Study 1: Smart Retail Inventory Management System

A retail chain implements a cloud-based inventory system connected to store devices. Clients (POS systems and dashboards) communicate via APIs to centralized cloud services. Cloud storage stores product data, transaction logs, and analytics datasets. The system uses platform services for analytics and demand forecasting. Secure access is maintained through VPNs, firewalls, and API gateways. This solution integrates infrastructure, services, and web applications for efficient operations.

2. Case Study 2: Cloud-Based Document Collaboration System

An enterprise deploys a document collaboration tool similar to Google Docs. Users access the platform via browsers, enabling real-time editing and sharing. Documents are stored in distributed cloud storage with version control mechanisms. Web APIs enable synchronization across multiple clients and devices. Network infrastructure ensures low latency and high availability globally. Security includes encryption, identity management, and access permissions for collaboration.

3. Case Study 3: Online Food Ordering Platform

A startup builds a cloud-based food ordering system accessible via web browsers and mobile clients. Frontend clients interact with backend services through RESTful Web APIs hosted on a cloud platform. Cloud storage is used to store menu images, invoices, and customer data securely. The infrastructure uses virtual machines and managed Kubernetes for scalability and reliability. Security is enforced through authentication tokens, HTTPS, and role-based access control. The system demonstrates integration of clients, APIs, storage, and platform services in real time.

Case Study Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Case	25%	Clearly identifies all technical and business requirements	Identifies most requirements	Basic understanding only	Poor understanding of case
Application of Concepts	25%	Applies suitable cloud concepts and tools effectively	Good application with minor issues	Partial application	Weak or incorrect application
Analysis	20%	Strong analysis with clear trade-off reasoning	Reasonable analysis	Limited analysis	Minimal analysis
Solution Design	20%	Solution is feasible, practical, and well justified	Reasonable solution with minor gaps	Basic solution	Unclear solution
Clarity	10%	Well-structured and easy to follow	Mostly clear	Somewhat unclear	Poorly organized

Case Study 1: Smart Retail Inventory Management System

Overview

A retail chain deploys a cloud-based inventory management system that connects Point-of-Sale (POS) terminals and management dashboards to centralised cloud services. The architecture brings together cloud storage, web APIs, platform analytics services, and layered network security to support efficient, real-time retail operations.

1. Cloud Storage

Cloud storage forms the data backbone of the retail system and is used to persist three main categories of data:

Product Data – A structured object or relational store (e.g., Amazon S3 with metadata catalogues, or Azure Blob Storage paired with Azure SQL Database) holds the full product catalogue, including SKUs, pricing, stock levels, and supplier details. This data must be highly available and quickly readable by POS terminals across all branches.

Transaction Logs – Every sale, return, or stock adjustment generates an event record. These logs are written to append-optimised cloud storage (such as Amazon S3 or Azure Data Lake Storage) in near real time. Immutability policies (write-once, read-many) protect log integrity for auditing and compliance purposes.

Analytics Datasets – Aggregated and processed datasets derived from transaction logs are stored in a data warehouse layer (e.g., Google BigQuery, Amazon Redshift, or Azure Synapse Analytics). These datasets power dashboards and feed machine learning models for demand forecasting.

The storage tier benefits from redundancy across multiple availability zones, ensuring no single hardware failure can disrupt operations. Data lifecycle policies automatically archive older records to cheaper cold-storage tiers (e.g., S3 Glacier), reducing costs.

2. Web APIs

Web APIs are the communication bridge between clients (POS systems and dashboards) and the cloud services. The system employs RESTful APIs over HTTPS for most client interactions:

- **Inventory Query API** – POS terminals call this endpoint to check real-time stock levels before confirming a sale. The request/response cycle is lightweight (JSON payloads) and is optimised for low latency.
- **Transaction Recording API** – After each sale, the POS posts a transaction record to the cloud. The API validates the payload, writes to cloud storage, and responds with a confirmation.
- **Dashboard/Reporting API** – Management dashboards call reporting endpoints to retrieve aggregated sales data, stock movement trends, and low-stock alerts.

- **Demand Forecasting API** – An internal API wraps the machine learning forecasting service, allowing the inventory management application to request reorder recommendations without directly coupling to the ML platform.

An **API Gateway** (e.g., AWS API Gateway, Azure API Management) sits in front of these services to handle routing, rate limiting, request throttling, and authentication token validation. This single entry point simplifies client integration and enforces consistent security policies across all API traffic.

3. Platform Services

Beyond raw infrastructure, the system exploits managed cloud platform services that eliminate the need to build and maintain specialist systems from scratch:

Analytics Service – A managed analytics platform (such as Google Looker Studio, AWS QuickSight, or Azure Power BI Embedded) ingests the datasets from cloud storage and generates visual dashboards. Managers can track KPIs such as daily revenue, top-selling products, and inter-branch stock comparisons without writing any queries.

Demand Forecasting / Machine Learning Service – A managed ML service (e.g., AWS SageMaker, Azure Machine Learning, or Google Vertex AI) trains predictive models on historical transaction data. The trained model surfaces reorder recommendations: when stock of a product is projected to reach a reorder threshold given expected demand, the system automatically raises a purchase order. This reduces both stockouts and overstocking.

Notification / Messaging Service – A managed pub/sub or queue service (e.g., Amazon SNS/SQS, Azure Service Bus) decouples the transaction recording pipeline from downstream consumers. When a sale is recorded, an event is published; the analytics ingestion pipeline and the stock-level updater consume the event independently and asynchronously, improving resilience.

4. Network Security

The system enforces a layered "defence-in-depth" security model:

VPN (Virtual Private Network) – POS terminals in each store connect to the cloud environment over a site-to-site VPN tunnel. This encrypts all traffic in transit between store networks and the cloud, preventing interception on public or shared network segments.

Firewalls – Cloud-based firewalls (Network Security Groups in Azure, or Security Groups and NACLs in AWS) restrict inbound and outbound traffic to explicitly permitted ports and IP address ranges. POS terminals are only allowed to reach specific API endpoints; direct access to databases or storage buckets is blocked.

API Gateway – Acts as a secure, managed perimeter for all API traffic. It enforces HTTPS, validates authentication tokens (e.g., OAuth 2.0 bearer tokens or API keys), and rejects malformed or unauthorised requests before they reach backend services.

Identity and Access Management (IAM) – Cloud IAM roles and policies ensure that each service component (the analytics pipeline, the ML service, the transaction recorder) has only the minimum permissions required—the principle of least privilege.

Encryption – Data is encrypted at rest using platform-managed keys (AES-256) and encrypted in transit using TLS 1.2 or higher.

5. Architecture Summary

The architecture follows a three-tier model:

Tier	Components
Client	POS terminals, management dashboards
Services	API Gateway → REST APIs → Microservices (inventory, transactions, forecasting)
Data	Cloud object storage, relational DB, data warehouse, ML model store

This layered design means each tier can be scaled independently. During peak retail periods (e.g., holiday sales), the API and compute layer can auto-scale horizontally without requiring changes to the storage or client tiers.

Case Study 2: Cloud-Based Document Collaboration System

Overview

An enterprise deploys a real-time document collaboration platform analogous to Google Docs. Users access it via a browser, edit documents simultaneously with colleagues, and rely on the platform to manage version history, synchronise changes across devices, and enforce security policies. The architecture must prioritise low latency, global availability, and data consistency.

1. Cloud Storage

Documents and their associated metadata are stored in a distributed cloud storage architecture designed for high availability and consistency:

Document Object Storage – Each document's content and embedded assets (images, charts) are stored in cloud object storage (e.g., Amazon S3, Azure Blob Storage). Objects are replicated across multiple geographic regions to ensure data durability (typically 99.999999999% durability, the "eleven nines" standard).

Version Control / History – Every save or auto-save operation creates a new version object in storage. Version metadata (author, timestamp, change delta) is recorded in a NoSQL or time-series database (e.g., DynamoDB, Cosmos DB, or Firestore). Users can browse the version history and restore any earlier state. Delta-based versioning (storing only the changes between versions, not full copies) reduces storage costs significantly for large documents with frequent edits.

Metadata and Index Store – A fast, queryable store (e.g., Elasticsearch or a managed SQL database) holds document titles, ownership, sharing permissions, and search indices. This powers the search functionality and the sharing/permission management UI.

2. Web APIs and Real-Time Synchronisation

Two complementary API patterns are used to support both standard operations and real-time collaboration:

RESTful APIs handle non-real-time operations: creating a new document, fetching a document list, updating sharing permissions, and requesting a specific version. These are stateless, cacheable calls served through an API Gateway.

WebSocket / Server-Sent Events (SSE) enable real-time collaboration. When multiple users have the same document open, a persistent WebSocket connection is maintained between each browser client and a real-time collaboration server. When User A types a character, the change is sent over the WebSocket to the server, which broadcasts it to all other connected clients editing the same document.

Operational Transformation (OT) or Conflict-Free Replicated Data Types (CRDTs) are the algorithms underlying multi-user conflict resolution. When two users edit the same region of a document simultaneously, OT or CRDT logic determines the correct merged result without data loss — this is the core technical challenge that distinguishes collaborative editing from simple file sharing.

Sync API – A REST/HTTP2 fallback synchronisation API is used for clients that cannot maintain WebSocket connections (e.g., mobile clients on unreliable networks). It polls for recent deltas and applies them locally.

3. Network Infrastructure

Content Delivery Network (CDN) – Static assets (the application's JavaScript, CSS, icons, and read-only document previews) are served from CDN edge nodes located close to users worldwide. This dramatically reduces load times for the initial page render.

Global Load Balancing – DNS-based or anycast global load balancers route each user's connection to the nearest available data centre, minimising round-trip latency for real-time WebSocket messages.

Low-Latency Regional Clusters – Real-time collaboration servers are deployed in multiple regions. A user in Europe connects to a European cluster; a user in Asia connects to an Asian cluster. Inter-region

replication keeps document state consistent across clusters, but each user is served locally to keep latency under the ~100 ms threshold where humans perceive input lag.

High Availability – The platform runs across multiple availability zones within each region. Health checks and automatic failover ensure that if one zone or server fails, connections are transparently rerouted without interrupting the user's session.

4. Security

Encryption in Transit and at Rest – All data is encrypted using TLS 1.3 in transit and AES-256 at rest. Document content stored in object storage is encrypted with per-customer managed keys where enterprise tiers are offered.

Identity Management – Users authenticate via an Identity Provider (IdP) using OAuth 2.0 / OpenID Connect (OIDC). Enterprise customers can integrate their corporate directory (Active Directory, Okta, Google Workspace) via SAML 2.0 single sign-on (SSO), ensuring only authorised employees can access the platform.

Access Permissions – Each document has a granular permission model: Owner, Editor, Commenter, Viewer. The API enforces these permissions on every request. Sharing links can be scoped (e.g., "anyone with the link can view" vs. "specific people only") and can include expiry dates.

Audit Logging – All access events, permission changes, and document exports are written to immutable audit logs stored in cloud storage. This supports compliance requirements (GDPR, ISO 27001, etc.) and forensic investigation.

5. Architecture Summary

Layer	Components
Client	Browser (JavaScript SPA), mobile apps
API / Real-time	API Gateway (REST), WebSocket servers (collaboration), CDN
Application	Document service, versioning service, permission service, notification service
Data	Object storage (content), NoSQL (versions/metadata), Search index
Security	IdP/SSO, IAM, TLS, encryption at rest, audit logs

The distributed nature of the storage and the real-time synchronisation layer are the defining technical challenges of this system, solved through replication, CDN edge delivery, and conflict-resolution algorithms.

Case Study 3: Online Food Ordering Platform

Overview

A startup builds a cloud-native food ordering platform accessible via web browsers and mobile apps. Customers browse restaurants, place orders, and track deliveries. Restaurants manage menus and order queues. The architecture emphasises scalability (handling peak meal-time demand), reliability, and security, leveraging RESTful APIs, container orchestration, and managed cloud storage.

1. RESTful Web APIs

The frontend clients (browser and mobile) interact with backend services exclusively through RESTful Web APIs over HTTPS. The API design follows REST principles:

- **Resources are nouns** – /restaurants, /menus/{id}, /orders, /customers/{id}
- **HTTP verbs define actions** – GET to retrieve, POST to create, PUT/PATCH to update, DELETE to remove
- **Stateless** – Each request is self-contained; authentication state is carried in a token header, not server-side session

Key API endpoints in the platform:

Endpoint	Method	Purpose
/restaurants	GET	List available restaurants
/menus/{restaurantId}	GET	Fetch menu items and prices
/orders	POST	Place a new order
/orders/{id}	GET	Track order status
/customers/{id}	GET/PATCH	View/update customer profile
/payments	POST	Process payment

An **API Gateway** (e.g., Kong, AWS API Gateway, or GCP Apigee) sits in front of all t

hese services, providing a single endpoint for clients, routing requests to the appropriate microservice, enforcing rate limits, and validating authentication tokens.

2. Cloud Storage

Cloud storage serves three distinct data types in this platform:

Menu Images – High-resolution photographs of dishes and restaurant logos are stored in a cloud object store (e.g., S3, GCS). These assets are served to users via a CDN so that images load quickly regardless of the user's geographic location. Images are stored with public read access while write access is restricted to authenticated restaurant admin accounts.

Invoices – After each order is completed, a PDF invoice is generated server-side and stored in a private container in cloud object storage. A time-limited pre-signed URL is generated and sent to the customer for download. Invoices are retained for a defined period (e.g., 7 years) to satisfy tax/accounting compliance.

Customer Data – Structured customer records (name, email, delivery addresses, order history) are stored in a managed relational database (e.g., Amazon RDS, Azure Database for PostgreSQL) within a private VPC subnet. This data is classified as sensitive personal data and subject to data protection regulations (e.g., GDPR), driving requirements for encryption, access logging, and the right to erasure.

3. Infrastructure: Virtual Machines and Kubernetes

Virtual Machines (VMs) provide the underlying compute for certain stable, long-running workloads such as databases and caching layers (e.g., Redis for session caching and menu caching) where container orchestration adds unnecessary complexity.

Managed Kubernetes (e.g., AWS EKS, GKE, or AKS) is the core platform for deploying and scaling the microservices that make up the application backend. Kubernetes provides:

- **Horizontal Pod Autoscaling (HPA)** – During peak ordering times (e.g., 12–1 pm lunchtime rush), Kubernetes automatically spins up additional instances of the order service and payment service to handle the increased load, then scales back down in off-peak hours. This elasticity is a key advantage of cloud-based deployments over fixed on-premises infrastructure.
- **Self-healing** – If a container crashes or becomes unresponsive, Kubernetes automatically restarts it and reroutes traffic to healthy instances. This contributes to the platform's high availability.
- **Rolling Deployments** – New versions of the application are deployed with zero downtime using rolling update strategies; a fraction of containers are updated at a time while the rest continue serving traffic.
- **Service Discovery** – Kubernetes DNS allows microservices to find and communicate with each other by name (e.g., the order service calls `payment-service.default.svc.cluster.local`) without hardcoded IP addresses.

The combination of VMs for stateful workloads and Kubernetes for stateless microservices represents a pragmatic hybrid infrastructure approach.

4. Security

Authentication Tokens – Users authenticate using username/password or social login (OAuth 2.0). Upon successful authentication, the server issues a JSON Web Token (JWT). The JWT is signed with a private key and contains claims (user ID, roles, expiry). Every subsequent API request includes the JWT in the Authorization: Bearer <token> header. The API Gateway validates the signature and expiry on every request before forwarding it to a backend service.

HTTPS (TLS) – All communication between clients and the API Gateway is encrypted using TLS 1.2 or higher. Certificates are managed and auto-renewed using a service such as AWS Certificate Manager or Let's Encrypt. HTTP traffic is redirected to HTTPS at the load balancer level.

Role-Based Access Control (RBAC) – Three primary roles govern access:

Role	Permissions
Customer	Browse menus, place/track orders, manage own profile
Restaurant Admin	Manage own menu, view own orders, update order status
Platform Admin	Full access: user management, reporting, moderation

The API enforces RBAC by reading the role claim from the validated JWT and checking it against the required permission for the requested resource. A restaurant admin cannot access another restaurant's orders; a customer cannot modify menu items.

Network Segmentation – The Kubernetes cluster runs within a private VPC. Only the API Gateway and load balancer are exposed to the public internet. Database and internal service pods are in private subnets with no direct inbound internet access.

5. Architecture Summary

Layer	Components
Client	Web browser (React/Vue SPA), mobile app (iOS/Android)
Edge	CDN (static assets + menu images), WAF
API	API Gateway → REST microservices (order, menu, payment, user)
Compute	Kubernetes pods (microservices), VMs (database, cache)
Storage	Object storage (images, invoices), Managed RDS (customer/order data)
Security	JWT auth, HTTPS/TLS, RBAC, VPC segmentation, IAM

This architecture demonstrates how a startup can build a production-grade platform without operating physical infrastructure — every layer (compute, storage, networking, security) is provided and managed by the cloud provider, allowing the engineering team to focus on application logic rather than infrastructure operations.